

LabGuardian 项目报告

基于边缘 AI 的智能实验助教系统

——基于边缘 AI 的电子实验教学辅助系统最终报告

项目名称	LabGuardian：基于边缘 AI 的智能实验助教系统
系统构成	边缘推理服务；实验交互界面；知识增强诊断模块
报告用途	实验最终报告、项目验收材料、课程提交材料
撰写依据	项目任务要求、系统设计方案、实验过程记录与测试结果
版本说明	以已完成的系统设计、实现与验证工作为依据；后续可补充更大规模实测数据
日期	2026 年

说明：本报告用于说明 LabGuardian 系统的研究背景、总体设计、关键实现、测试验证与应用价值。

目录

摘要

第一章 项目概述

第二章 项目背景与需求分析

第三章 总体设计方案

第四章 服务端系统设计与实现

第五章 交互端系统设计与实现

第六章 数据结构与接口设计

第七章 关键技术与创新点

第八章 测试验证与质量保障

第九章 部署运行与边缘化方案

第十章 项目完成情况与边界说明

第十一章 风险分析与改进计划

第十二章 应用价值、推广路径与实施组织

第十三章 结论

参考资料

附录：术语与模块说明

注：本目录为静态目录，正式排版时可根据最终页码统一更新。

摘要

LabGuardian 是一套面向高校电子类基础实验的智能实验助教系统。系统以面包板电路搭建、元器件引脚定位、孔位映射、拓扑重构、参考电路比对和诊断解释为主线，针对传统实验教学中教师巡检压力大、学生排错效率低、真实电路遮挡严重、原理图与实物连接映射困难等问题，提出面向真实实验过程的边缘智能辅助方案。系统面向 DK-2500 等边缘计算平台，强调本地化推理、知识检索、异构计算和教学数据隐私保护，并形成了由视觉感知、结构化网表、逻辑比较、PCM Agent、交互界面和硬件遥测组成的完整实验闭环。

本报告围绕项目目标、需求分析、总体架构、系统实现、接口数据结构、关键技术、测试验证、部署方案和后续改进方向展开。系统主链路聚焦面包板电路的结构化诊断，采用 S1 组件检测、S1.5 全图引脚姿态检测、S2 孔位映射、S3 拓扑与 netlist_v2 生成、S4 逻辑参考比较、S5 语义分析的事实链；PCM Agent 以结构化证据和确定性工具为基础生成解释，避免将事实判断直接交由大模型完成。PCB AOI、端侧 VLM 微观缺陷识别等能力作为后续扩展方向，不作为本报告已完成主链路的核心内容。

关键词：边缘 AI；电子实验教学；面包板电路；YOLO-Pose；BoardSchema；netlist_v2；图同构比较；PCM Agent；RAG 知识增强；React 交互端。

Abstract

LabGuardian is an intelligent tutoring system for fundamental electronic laboratory courses in higher education. The system focuses on breadboard circuit construction, component pin localization, hole mapping, topology reconstruction, reference circuit comparison, and diagnostic explanation. It addresses practical problems in traditional electronics labs, including heavy teacher inspection workload, low troubleshooting efficiency for students, severe occlusion in real circuit images, and the difficulty of mapping schematics to physical connections. Targeting edge computing platforms such as DK-2500, the system emphasizes local inference, knowledge retrieval, heterogeneous computing, and privacy protection for teaching data. It integrates visual perception, structured netlist representation, logical comparison, PCM Agent, interactive interface, and hardware telemetry into a complete experimental workflow.

This report systematically summarizes the project goals, requirements, overall architecture, system implementation, interface data structures, key technologies, testing and deployment plans, project progress, and future improvement directions. The main workflow focuses on structured diagnosis of breadboard circuits, following the fact chain of S1 component detection, S1.5 full-image pin pose detection, S2 hole mapping, S3 topology and netlist_v2 generation, S4 logical reference comparison, and S5 semantic analysis. The PCM Agent generates explanations based on structured evidence and deterministic tools instead of delegating factual judgment to a large language model. PCB AOI and edge-side VLM micro-defect recognition are treated as future extensions rather than completed core functions.

Keywords: edge AI; electronic laboratory teaching; breadboard circuit; YOLO-Pose; BoardSchema; netlist_v2; graph isomorphism comparison; PCM Agent; RAG knowledge enhancement; interactive interface.

第一章 项目概述

1.1 项目定位

LabGuardian 的项目定位是“面向电子实验教学场景的边缘智能助教系统”。与通用电路仿真软件不同，它不是只在虚拟环境中验证电路原理，而是从学生真实搭建的面板照片出发，将图像中的元件、引脚、孔位和电气连接转换为可验证、可审计的结构化事实，再与教师给出的参考电路进行逻辑比较，最后向学生输出错误位置、错误原因、风险等级和修改建议。系统的核心价值在于把实验教学中的经验性巡检过程数字化、结构化和可解释化，使教师能够从重复排错中解放出来，学生也能在搭建过程中获得及时反馈。

系统主线可概括为“`component_id + pin_name + hole_id` → `electrical_node_id` → `electrical_net_id` → `netlist_v2`”的结构化事实链。该链路把视觉识别结果与电气语义连接起来：`component_id` 表示识别出的元件实例，`pin_name` 表示元件具体引脚，`hole_id` 表示物理面包板孔位，`electrical_node_id` 表示面包板静态导通节点，`electrical_net_id` 表示由导线和元件连接合并后的电气网络。只要这条链路可靠，系统就能把“图片中看起来接在某处”的模糊观察转化为“该引脚属于哪个电气网络”的确定性判断。

1.2 建设目标

- 构建从图像上传、元件识别、引脚检测、孔位映射、拓扑重构到诊断解释的完整实验链路。
- 形成稳定的数据契约，使交互端、服务端、Agent、知识库和测试用例都围绕同一套结构化事实运行。
- 面向电子实验教学输出可解释结果，包括错误码、证据引用、建议动作、风险等级和交互端高亮目标。
- 为 DK-2500 等边缘设备部署预留模型路径、运行元数据、硬件遥测和后续 OpenVINO/INT8 优化入口。
- 在系统架构上坚持“事实判断由确定性算法完成，Agent 负责解释与引导”，降低大模型幻觉风险。

1.3 报告范围与依据

本报告覆盖项目背景、需求分析、总体方案、系统实现、接口和数据结构、关键技术、测试部署、完成情况和改进计划。报告以项目任务要求、实验过程记录、系统设计材料和阶段性测试结果为依据，重点说明系统的设计逻辑、实现边界和验证方式。

需要说明的是，PCB 无监督 AOI、Anomalib 热力图和 NPU VLM 微观诊断等内容属于面向后续阶段的扩展方向。本报告将其纳入“后续拓展与风险控制”讨论，不将其表述为当前主链路已经完成的能力，以保证报告内容与实际系统边界一致。

第二章 项目背景与需求分析

2.1 教学背景

高校电子类基础实验通常覆盖电路分析、模拟电子技术、数字电子技术、电子工艺实训等课程，是学生从理论学习走向工程实践的重要环节。传统课堂中，学生需要依据原理图在面包板上插接电阻、电容、二极管、LED、晶体管、运放或其他集成芯片，并通过电源、示波器、万用表等仪器观察实验现象。这个过程本质上需要学生完成三个映射：从抽象电路原理映射到元件功能，从二维原理图映射到面包板孔位，从实验现象映射到具体故障原因。对初学者而言，任何一个映射失败都可能导致实验停滞。

在实际教学组织中，电子实验通常存在较高的师生比，教师难以在同一时间覆盖所有实验台。学生遇到问题后，往往需要排队等待教师检查；教师到场后也需要从“元件是否插对、引脚是否接反、孔位是否同一导通组、电源轨是否误接、导线是否短路”等基础问题逐项排查。这类排查高度依赖经验，重复性强，但又不能简单省略，因为极性反接和电源短路会造成芯片损坏甚至实验安全风险。

2.2 核心痛点

第一，真实面包板图像的遮挡问题明显。面包板上导线常常互相交叠，元件本体可能遮住引脚，拍摄角度、反光、阴影和焦距都会影响视觉识别。传统目标检测框只能告诉系统“这里可能有一个元件”，但难以准确给出引脚插入了哪个孔位。对于电子诊断而言，元件框级别的识别远远不够，系统必须进入引脚级和孔位级。

第二，学生对面板静态导通关系不熟悉。面包板主区域通常按行导通，左右半区又被中缝隔开，电源轨还可能存在分段。学生可能认为两个孔在图像上很近就等价，或者忽视 A-E 与 F-J 的隔离关系。系统如果能将 `hole_id` 映射为 `electrical_node_id`，就可以明确告诉学生“当前位置虽然相邻，但不在同一导通节点”或“这两个引脚实际上落在了同一网络，存在短路风险”。

第三，教师需要可解释证据而不是黑盒结论。实验教学场景中的 AI 诊断不能只输出“电路错误”四个字，而应说明错误码、涉及的元件、引脚、孔位、参考连接、当前连接以及建议动作。LabGuardian 当前的 `validator_report_v2` 就围绕这一点设计，它在每条诊断项中保留 `evidence_refs`，并且可被交互端转换为框选元件、点亮引脚、标注孔位的高亮协议。

第四，系统需要适应边缘环境。高校实验室网络条件并不总是稳定，课堂图像和学生实验数据也不宜全部上传云端。项目选择边缘设备作为主要运行平台，既能降低依赖外部网络的风险，也有利于现场教学和验收展示。系统统一了模型目录、推理参数和运行元数据记录方式，为后续 DK-2500 真机部署和性能评测提供基础。

2.3 用户角色需求

角色	主要需求	系统响应
学生	快速知道电路哪里错、为什么错、如何改；能上传图片并看到可视化定位。	交互端上传图片，显示识别框、孔位、网表、诊断卡片和 Agent 建议。

教师/助教	了解多个实验台状态，定位共性错误，减少重复巡检。	服务端维护 ClassroomState，记录工位风险、诊断、网表和缩略图，支持教师看板扩展。
系统维护者	保持视觉、拓扑、比较和 Agent 模块的数据契约稳定，便于后续维护与扩展。	通过结构化接口、测试样例和版本记录约束各阶段输入输出。
评审/验收人员	关注端到端链路、边缘部署价值、可解释诊断和系统完整性。	系统展示从图像输入到事实链生成、诊断解释和运行观测的完整过程。

2.4 功能需求

- 图像接入：支持上传 1 至 3 张 base64 编码的面包板图片，当前交互端默认上传一张主视角图片，服务端协议可扩展多视角。
- 组件检测：识别电阻、电容、导线、LED、二极管、三极管、电位器、IC 等教学常见元件，生成全局 component_id。
- 引脚检测：通过全图 YOLO-Pose 或几何规则输出 ordered pins，并保留多视图观测、置信度和来源信息。
- 孔位映射：将引脚关键点吸附到面包板 hole_id，并进一步映射到 electrical_node_id。
- 拓扑重构：基于 BoardSchema 和 CircuitAnalyzer 合并导线网络，输出 netlist_v2、拓扑图和 SPICE 网表。
- 参考比较：将当前 netlist_v2 与 logical_reference_v1 参考电路转换为图模型，执行拓扑匹配、角色推断和差异报告。
- 诊断解释：把 validator_report_v2、RuntimeEvidence、ContextPack 和工具结果转化为学生可理解的自然语言建议。
- 人工修正：交互端支持端口标注、孔位修正、网络角色指定、IC 标注和引脚极性选择，提交后重跑 S3/S4。
- 可视化：交互端显示阶段进度、元件框、引脚点、孔位高亮、网表表格、诊断项和原始 JSON。
- 运行观测：服务端支持 runtime_metadata，并提供 CPU、内存、iGPU、NPU 相关硬件遥测协议。

2.5 非功能需求

非功能需求主要包括准确性、可解释性、低延迟、弱网可用、隐私保护、可维护性和可扩展性。准确性体现在视觉识别、孔位吸附、导通节点映射和逻辑比较的综合正确率；可解释性体现在每个错误都有证据引用和可视化目标；低延迟要求端到端推理在课堂交互中足够及时；弱网和隐私要求系统尽量本地运行；可维护性要求不同阶段协议清晰、测试覆盖充分；可扩展性要求新增板型、元件、参考电路、故障案例和教学知识时不需要重写核心链路。

第三章 总体设计方案

3.1 总体架构

LabGuardian 采用交互端与服务端分离的系统架构。交互端负责图片上传、参数选择、参考电路选择、结果可视化、人工修正和 Agent 对话；服务端负责接口接入、图像处理、模型推理、孔位映射、拓扑构建、逻辑比较、课堂状态、知识检索、Agent 编排和遥测推流。整体架构可以概括为“Web 交互层—API 服务层—Pipeline 事实层—Domain 规则层—Agent/Knowledge 解释层—Infra 支撑层”。

系统服务端可划分为 FastAPI/WebSocket API、Services、Agent/Knowledge、Domain、Pipeline 和 Infra 六层。API 层作为协议入口，不承担领域推理；Services 层负责编排、审计、下发和任务管理；Domain 层承载 BoardSchema、CircuitAnalyzer、validator、risk、reference DSL 等稳定规则；Pipeline 层只输出结构化事实，不直接生成教学话术；Agent/Knowledge 层在事实基础上组织上下文、工具调用和回答；Infra 层提供异步任务、缓存、容器化和测试支撑。

层级	主要模块	职责说明
Web 交互层	React、Vite、TypeScript、结果画布、网表视图、Agent 对话	完成图片上传、参数配置、结果展示、人工修正、Agent 交互和流程编排。
API 层	Pipeline 接口、Agent 接口、课堂状态接口、WebSocket 与遥测接口	提供同步/异步 Pipeline、Agent 问答、课堂状态、遥测推流等接口。
服务层	流水线服务、指导服务、课堂状态、参考电路服务、Agent 服务	负责业务编排、结果封装、参考电路解析、课堂态同步和任务状态管理。
Pipeline 层	流程编排、组件检测、引脚检测、孔位映射、拓扑重构、参考验证、语义分析	从图像输入到诊断事实输出的主链路。
Domain 层	板型规则、电路分析、网表模型、逻辑参考、图比较、风险分类	提供板型、电气网络、参考电路、图比较和风险分类等确定性规则。
Agent/Knowledge 层	运行证据、上下文包、确定性工具、状态机、回答验证、芯片/电路/故障知识库	把结构化事实转化为可控上下文和教学解释。

3.2 主业务流程

一次完整诊断流程从交互端上传图片开始。用户选择参考电路、设定置信度、IoU、推理尺寸和电源轨角色后，交互端将图片转为纯 base64 字符串，提交到 POST /api/v1/pipeline/run。服务端 PipelineService 解析 reference_id 或 inline reference_circuit，调用 orchestrator 执行 S1 到 S5。执行完成后，服务层将结果封装为 PipelineResult，同时同步到 ClassroomState。交互端接收结果后显示识别事实链和诊断报告，并自动向 Agent 提交“根据当前诊断结果给出诊断解释和下一步建议”的提示，Agent 再基于 ClassroomState 中的 RuntimeEvidence 生成自然语言解释。

如果学生或教师发现视觉映射结果不准确，交互端可以进入网表模式进行人工修正。用户可以修正引脚孔位，给输入/输出端口打标，设置网络角色，或补充三极管和 IC 的人工标注。交互端随后调用 `/api/v1/pipeline/recompute-corrected`，把 mapping components 与修正 patch 提交给服务端。服务端不再重跑视觉检测，而是从修正后的 components 重新执行 S3 拓扑、S4 验证和 S5 语义分析。这种设计兼顾 AI 自动识别与教学现场的人机协同，避免视觉阶段偶发错误导致整个系统不可用。

3.3 技术选型

类别	选型	理由
服务端框架	FastAPI、Pydantic、Uvicorn	接口实现效率高，类型契约清晰，适合返回结构化 JSON。
异步任务	Celery、Redis	支持较长模型推理任务异步提交与状态查询，同时保留同步接口以满足课堂快速验证。
视觉算法	Ultralytics YOLO-Detect、YOLO-Pose、OpenCV	覆盖元件检测和引脚关键点检测，便于训练和部署。
拓扑建模	NetworkX、Union-Find、BoardSchema	适合构建二分图、合并导通网络、输出可解释网表。
知识/Agent	PCM ContextPack、deterministic tools、LangGraph、Datasheet/Circuit KB	把事实、工具、回答分层，降低大模型幻觉风险。
交互端	React 19、Vite、TypeScript、lucide-react	适合构建交互式单页应用和课堂看板。
边缘部署	Docker、OpenVINO/GenAI 预留、runtime_metadata、telemetry	为 DK-2500 本地推理和性能评测做准备。

3.4 设计原则

系统设计遵循四项原则。第一，事实链优先。视觉模型只产生候选事实，最终教学结论必须经过 BoardSchema、拓扑分析和 validator 约束。第二，接口契约优先。S1、S1.5、S2、netlist_v2、validator_report_v2、RuntimeEvidence 和 ContextPack 都以结构化 schema 作为协作边界，避免交互端、服务端或 Agent 私自猜测。第三，人机协同优先。系统允许人工端口标注和孔位修正，教师与学生可以把 AI 输出修正为可信事实后再重算。第四，边缘可观测优先。运行结果保留模型路径、推理参数、板型 schema、版本号和阶段耗时，后续可与遥测数据一起用于验收展示和实验分析。

第四章 服务端系统设计与实现

4.1 服务端模块与职责划分

服务端按照接口接入、基础配置、智能诊断、领域规则、视觉流水线、数据模型、业务服务和异步任务等模块组织。接口模块负责接收 Pipeline、Agent、课堂状态和遥测请求；基础模块负责运行配置、依赖管理和异步任务；领域模块承载电气拓扑、板型规则、参考电路、比较器和风险规则；流水线模块负责从视觉输入到验证诊断的阶段化处理；业务服务模块负责结果封装、课堂状态同步和任务调度。各模块通过结构化数据契约连接，以降低模块耦合并提高后续维护性。

模块	作用	报告中对应内容
Pipeline 编排模块	串联 S1→S1.5→S2→S3→S4→S5，管理共享模型和进度回调。	Pipeline 主流程。
组件检测模块	组件检测，top 主实例，side 候选补召回。	视觉组件检测。
引脚检测模块	全图 YOLO-Pose 引脚检测，关联回组件。	引脚关键点识别。
孔位映射模块	关键点到 hole_id/electrical_node_id 的映射和多视图投票。	孔位映射。
电气拓扑模块	CircuitAnalyzer、Union-Find、电气网络、SPICE/netlist 导出。	拓扑重构。
参验证模块	logical_reference_v1 比较和 validator_report_v2 输出。	验证诊断。
上下文构建模块	错误族路由、工具白名单、上下文包和指标估算。	PCM Agent。
Agent 编排模块	ReAct + Self-Reflection 状态机与顺序 fallback。	Agent 编排。

4.2 Pipeline 编排

Pipeline Orchestrator 是服务端事实链的核心。它通过线程安全单例复用 ComponentDetector 和 PinRoiDetector，避免 Celery worker 或同步请求每次都重新加载模型；同时为每次请求创建独立 BreadboardCalibrator，因为校准器包含当前图片的网格状态，不能跨请求共享。run_pipeline 接收 images_b64、reference_circuit、rail_assignments、port_annotations、net_role_assignments、net_alias_assignments、net_merge_assignments 以及 conf、iou、imgsz 等参数，依次执行检测、引脚检测、映射、拓扑、验证和语义分析，并返回 stages、total_duration_ms 与 runtime_metadata。

默认电源轨配置体现出运放实验场景需求：top_plus 为 VCC，top_minus 为 VEE，bot_plus 与 bot_minus 为 GND；用户可以在请求中覆盖这些设置。S3 后系统会应用端口和网络角色标注，并通过 normalize_current_netlist 添加逻辑名、别名和合并信息。S4 使用修正后的 netlist_v2 进行参考比

较。运行结束时，`runtime_metadata` 会记录系统版本、模型版本、知识库版本、规则版本、模型目录、模型路径、设备、置信度、IoU、推理尺寸和板型参数，为后续性能评估和问题追溯提供依据。

4.3 S1：组件检测

S1 阶段输入 1 至 3 张 base64 JPEG 图片，输出 `top` 主实例和 `side` 补召回候选。系统采用主视图优先策略：`top` 视图负责建立全局 `component_id`，侧视图只负责 `supplemental_detections`，不直接进入主实例列表。这一设计能够避免多视角下同一元件实例融合时出现重复或错配；先用 `top` 建立主事实，再把 `side` 作为候选证据，可以降低误合并风险。

组件检测返回 `component_detect_v1` 格式结果，记录检测器类型、契约版本、主检测结果、补召回结果、图像尺寸、解码统计和阶段耗时等信息。每个检测项包含元件编号、元件类别、封装类型、引脚模型、置信度、边界框、方向、视角编号、导线颜色和兼容字段等。系统会过滤 `Breadboard`、`Line_area` 等背景类，并对暂不支持的类别显式记录，避免“模型有输出但下游无结果”的问题被静默掩盖。对于 IC 元件，S1 还会通过封装识别逻辑推断 DIP8、DIP14 等 `package` 信息，为后续引脚生成提供依据。

4.4 S1.5：全图引脚检测

S1.5 阶段是从元件框级识别进入引脚级事实的关键。当前正式主路径不再对单个组件裁剪 ROI 后分别识别引脚，而是对 `top` 整图执行 `full-image YOLO-Pose`，再根据类别和 `bbox` 几何关系将 `pose` 实例关联回 S1 组件。这种方式能减少 ROI 裁剪误差和局部上下文丢失，也更符合未来多视角融合的证据整理需求。

引脚检测阶段会先判断 `top` 图像是否可用、引脚检测方式是否为 `yolo_pose`、模型是否已加载。满足条件时进入 `full_image_model` 路径，输出 `component_pin_detect_v1`；否则输出 `schema-compatible` 的 `unavailable` 外壳，保证下游阶段不会因为模型缺失而结构崩溃。对于 IC 元件，系统可以不完全依赖引脚模型，而是根据 `bbox` 与面包板行列约束生成 8 或 14 个引脚，再交给 S2 完成最终 `hole_id` 映射。每个 `pin` 记录 `pin_id`、`pin_name`、`keypoints_by_view`、`visibility_by_view`、`score_by_view`、`source_by_view`、`confidence`、`source` 和 `metadata`，为 S2 提供完整证据。

4.5 S2：孔位映射与多视图证据融合

S2 阶段将 S1.5 输出的 `ordered pin predictions` 映射到面包板 `hole_id` 和 `electrical_node_id`。它首先尝试用 `top` 图像对 `BreadboardCalibrator` 做校准；如果校准失败，则回退到 `synthetic grid`，并在 `calibration.mode` 中显式标记。随后，系统对每个 `pin` 构造 `observations`，根据 `keypoint`、可见性、分数、来源和投影信息生成候选孔位，调用 `BoardSchema` 将 `hole_id` 解析为 `electrical_node_id`。

S2 的一个重要特点是保留不确定性，而不是把不确定性隐藏。每个 `mapped pin` 会包含 `candidate_hole_ids`、`candidate_node_ids`、`observation_count`、`visible_view_ids`、`is_ambiguous`、`ambiguity_reasons`、`evidence_source`、`decisive_view_id`、`fusion_confidence`、`fusion_margin`、`cross_view_agreement`、`snap_distance_px` 和 `snap_confidence`。多视图融合元数据还能说明每个视图的贡献、各视图 `top1` 候选、遮挡时的权重提升和最终选择来源。

孔位吸附质量由距离和网格 `pitch` 共同决定，`snap_confidence` 使用 $1-(d/pitch)^2$ 的二次衰减公式。模型置信度高但关键点远离孔位的结果会被自动降权，低于阈值时进入 `ambiguity_reasons`。对于两脚轴向元件、电解电容、跳线和电位器，S2 还实现了 `pair selector` 和几何先验，防止两个引脚被独立吸附到不符合元件物理形态的孔位。

4.6 S3：拓扑重构与 netlist_v2 生成

S3 阶段读取 S2 映射后的 `components[.pins[]`，通过 `build_analyzer_from_components` 构造 `CircuitAnalyzer`。`CircuitAnalyzer` 以 `NetworkX` 图表示元件与网络关系，并使用 `Union-Find` 合并由 `Wire` 连接的等电位网络。面包板静态导通关系由 `BoardSchema` 提供，`hole_id` 是 source of truth，系统会根据 `hole_id` 重新解析 `electrical_node_id`，避免上游修正 `hole_id` 后旧 `node_id` 残留导致错误。

在内部图中，元件节点和网络节点构成二分图；每条边携带 `pin`、`role`、`pin_role`、`component`、`type` 和 `hole_id` 等属性。对于 `Wire` 元件，系统不把它作为普通电路元件进入最终拓扑，而是通过 `Union-Find` 把两端网络合并。`build_topology_graph` 会将合并后的网络组分配为 `N0`、`N1` 等 net ID，并对非 `Wire` 元件建立 `component-net` 边。若某个非导线元件的多个引脚落入同一个网络，则会标记 `same_net`，用于短路风险判断。

S3 输出包括 `circuit_description`、`netlist_v2`、`normalized_components`、`topology_graph`、`component_count` 和 `duration_ms`。`netlist_v2` 保留 `components`、`pins`、`nets`、`node_index` 和 `board_topology`；其中 `board_topology` 含 `schema_id`、`board_type`、`node_to_holes` 和当前 `rail_assignments`，便于交互界面在用户拖拽或高亮时点亮完整导通条。系统还支持导出 SPICE 网表，`Wire` 通过 `Union-Find` 隐式合并，不在 SPICE 网表中单独出现。

4.7 S4：参考电路比较与诊断报告

S4 阶段只支持 `logical_reference_v1` 格式，不再支持孔位级或物理点位级 `reference` 对比。该设计符合教学场景：学生实际摆放孔位可以不同，但只要电气逻辑等价，电路应被判定为正确；相反，即使孔位看似接近，只要逻辑网络错误，就应输出诊断。S4 会把 `reference_circuit` 转换为 `reference graph`，把当前 `netlist_v2` 转换为 `current graph`，然后调用 `compare_logical_graphs`。

比较器的设计目标是让用户只标注输入/输出端口和电源轨，内部 `signal`、对称端口和无极性两脚器件顺序由系统推断。比较流程包括自动检测 `reference symmetry`、尝试完整图同构、失败时根据 `reference` 推断 `current net role`、判断 `current` 是否包含 `reference` 或为 `reference` 子图，最后才使用 `graph edit distance` 或 `fallback` 生成错接报告。匹配规则中，`power/ground` 严格匹配；`Resistor`、`Capacitor`、`Wire` 等无极性两脚器件忽略引脚顺序；`LED`、`Diode`、电解电容、三极管、电位器等功能引脚严格匹配。

S4 输出 `validator_report_v2` 兼容报告。每条诊断项包含 `error_code`、`category`、`severity`、`message`、`suggested_action`、`evidence_refs`、`component_id`、`pin_name`、`current_hole_id`、`target_hole_id`、`current_node_id`、`target_node_id`、`expected` 和 `actual` 等字段。常见错误码包括 `FLOATING_PIN`、`WIRE_ENDPOINT_UNCONNECTED`、`COMPONENT_SHORTED_SAME_NET`、`POWER_RAIL_SHORT`、`UNEXPECTED_NET_BRIDGE`、`HOLE_MISMATCH`、`POLARITY_REVERSED`、`COMPONENT_MISSING`、`PIN_MISSING`、`LED_SERIES_RESISTOR_MISSING` 等。交互端可利用 `evidence_refs` 和 `highlight_protocol` 显示具体错误位置。

4.8 服务层与课堂状态

`PipelineService` 是服务端业务编排的入口之一。`run_sync` 会根据 `reference_id`、inline `reference` 或默认配置解析参考电路，调用 `run_pipeline`，并把 `reference` 来源写入 `runtime_metadata`。结果构造为 `PipelineResult` 后，`sync_result_to_classroom` 会将 `component_count`、`net_count`、`progress`、`similarity`、

diagnostics、comparison_report、risk_level、risk_reasons、circuit_snapshot、netlist_v2、semantic_analysis、runtime_metadata 等信息写入 ClassroomState，同时缓存缩略图。

一个值得注意的设计是 topology_label 的写入。PipelineService 会尝试调用 GNN-A 拓扑分类器，从 netlist_v2 中识别当前实验场景；只有在分类器启用、存在共识、且置信度为 high 或 medium 时，才把 topology_label 写入 station。否则返回空字符串，让 scene_resolver 进入 fail-open 状态，不会静默默认到错误场景。这是检索契约中的重要防御机制，避免 Agent 在错误场景下检索不相关 fault_case。

4.9 PCM Agent 与知识检索

Agent 架构的核心思想是 PCM，即 Push-Based Context Management。它不是让大模型直接读取全部网表和全部知识库，也不是让大模型重新判断电路事实，而是先从 ClassroomState 中抽取 RuntimeEvidence，再根据 error_code、error_tag、risk_level、current_scene_id 和用户问题构建最小上下文包。ContextPack 包含 pushed_facts、allowed_tools、prompt_rules、citation_requirements、evidence_refs 和历史摘要，并计算字符数和估算 token 数，用于控制边缘端上下文规模。

错误族路由把 COMPONENT_SHORTED_SAME_NET 映射为 short_circuit，把 NODE_MISMATCH、HOLE_MISMATCH、FLOATING_PIN 映射为 wiring_mismatch，把 POLARITY_REVERSED 映射为 polarity_error，把 LED_SERIES_RESISTOR_MISSING 映射为 missing_protection，把缺元件和缺引脚映射到 missing_component 或 incomplete_circuit。不同错误族会得到不同工具白名单：短路场景需要 netlist_trace_tool、board_schema_lookup_tool、safety_rule_lookup_tool 和 heatmap_overlay_tool；接线错误需要 board_schema 与 netlist trace；极性错误需要 datasheet_lookup_tool；概念问答可启用 teaching_concept_lookup_tool，必要时再启用 datasheet 或 circuit KB。

LangGraph 只承担编排职责。系统按照 classify_error → build_context_pack → react_plan → react_observe → react_reflect 的流程组织 ReAct 循环，再进入 verify_answer；验证失败时执行 repair_answer，最后进入 finalize_answer。即使缺少图编排环境，系统也保留顺序 fallback。Verifier 要求回答包含当前错误码或 evidence_ref；如果 risk_level 为 danger，还必须包含断电、电源或短路复查提示。这种设计保障了回答的可审计性和安全性。

检索契约进一步规定 production agent graph 和蒸馏管线中合法知识源只有 teaching_scene、fault_case、datasheet_v2 和 circuit_kb，另有 station_state、pipeline_snapshot、reference_circuit 等结构化事实通道。旧的 KbService、旧 Chroma/PDF 库和 RagService.answer_with_kb 被禁止进入 Agent 主链路，避免训练部署不一致、云依赖和 wrong-scene 污染。

4.10 边缘部署与硬件遥测

边缘部署方案统一了模型路径和运行默认值。系统以 LABGUARDIAN_MODEL_ROOT 作为模型根目录，组件检测和引脚检测模型分别按预设候选路径查找。默认推理尺寸统一为 960，容器环境可将模型目录以只读方式挂载，并通过 YOLO_MODEL_PATH 与 PIN_MODEL_PATH 指定模型。

硬件遥测协议面向 DK-2500 运行场景，提供 /ws/telemetry/system 作为 5Hz WebSocket 主通道，/api/v1/telemetry/latest 作为 REST smoke 接口。telemetry_frame_v1 包含 ts、cpu_pct、mem_used_mb、mem_total_mb、igpu_pct、igpu_freq_mhz、npu_pct、npu_power_mw、pipeline_stage 和 sampler_status。采样器应能在硬件不可用或容器缺少 sysfs 时降级返回 null，不阻塞主业务。未来可将 pipeline 阶段耗时与 CPU/iGPU/NPU 占用曲线叠加，用于现场验收展示和实验分析。

第五章 交互端系统设计与实现

5.1 交互端总体定位

交互界面采用 React + Vite + TypeScript 实现单工位完整诊断流程。它的主要任务不是承担复杂业务规则，而是把服务端生成的结构化事实清晰呈现给学生和教师。典型使用主线为：上传面包板图片，执行 Pipeline 诊断，展示 S1-S4 事实链、元件、引脚、孔位和网表，再调用 Agent 展示诊断解释和修改建议。

交互端使用 Vite、TypeScript、React 和 lucide-react 图标库构建。系统界面按接口封装、类型定义、通用组件、诊断流程和样式模块组织：接口封装负责访问服务端能力，类型定义约束 Pipeline、Agent 和 UI 数据结构，通用组件实现上传、画布标注、阶段耗时、诊断卡片、网表和原始数据展示，诊断流程模块负责状态管理、钩子调用和主流程编排。

5.2 API 封装

接口客户端提供统一的 JSON 请求封装。它从环境配置读取服务端地址，默认请求超时 5 分钟，并使用 AbortController 控制超时；当响应失败时，系统解析 detail 字段生成结构化错误信息。该设计保证 Pipeline 推理较慢时交互端不会过早失败，也能向用户显示“请求超时，请确认服务端服务和模型推理状态”等可理解提示。

Pipeline 接口封装健康检查、版本查询、同步诊断、人工修正重算、电路分析和端口可视化等能力；Agent 接口封装问答提交、状态查询和结果轮询逻辑。核心服务契约包括健康检查、版本查询、Pipeline 运行、Agent 问答和 Agent 状态查询。

5.3 主页面功能组织

主页面是交互端诊断流程的中心。页面状态由统一状态管理逻辑维护，并围绕服务状态检查、参考电路选择、Pipeline 执行、Agent 对话和拓扑建议等功能组织。页面顶层包括系统状态栏、指标概览、上传面板、参考电路选择、证据链主区域、诊断面板、Agent 对话区、阶段时间线和原始数据查看区。

主区域有两种展示方式：网表模式支持孔位修正、端口标注、网络角色、引脚极性和 IC 标注；图像模式在原始图片上渲染识别框和高亮目标。诊断面板按严重程度对 comparison_report.items 排序，用户选择某条诊断后，系统从 evidence_refs 中提取高亮目标并传递给图像或网表视图，实现“点击错误—定位元件/引脚/孔位”的交互。

5.4 上传、运行与阶段进度

上传流程会先检查文件类型是否为 image/*，再将图像转换为 base64 字符串并保存到页面状态中。Pipeline 执行时提交 station_id、images_b64、conf、iou、imgsz、reference_id、rail_assignments、port_annotations、net_role_assignments 和 ic_annotations。运行成功后，交互端保存 PipelineResult，并自动调用 Agent 生成诊断解释。

为了提升使用体验，交互端为 detect、pin_detect、mapping、topology、validate、semantic_analysis 等阶段设置了前台进度节奏。这一节奏用于在请求过程中展示阶段推进，并不代表服务端真实耗时。真实耗时仍应以服务端 stages[].duration_ms、total_duration_ms 和 runtime_metadata 为准。

5.5 人工修正与重算

人工修正是交互端的重要能力。系统会从当前 PipelineResult 的 mapping 阶段取出 components，结合人工修正生成 correction patch，再整理 portAnnotations、manualNetRoleAssignments、manualIcAnnotations 和 pinPolarityAssignments。如果没有任何修正或标注，交互端会提示用户先标注端口、修改孔位或指定网络角色。若存在有效修改，则调用 recomputeCorrected，把这些结构化输入提交给服务端。

这种重算机制让系统从一次性自动判断转向“可纠错的教学工具”。在真实课堂中，视觉模型难免出现漏检、误检或孔位吸附偏差。如果系统不能允许人工修正，教师就只能放弃这次结果；而 LabGuardian 允许保留视觉阶段识别到的大部分事实，只对有争议的 pin 或端口做局部修改，再让拓扑和比较模块重新判断，这更符合实验教学中的人机协同逻辑。

第六章 数据结构与接口设计

6.1 主要接口

接口	方法	用途
/health	GET	交互端启动时检查服务端服务是否在线。
/version	GET	获取系统版本、模型版本、规则版本等展示信息。
/api/v1/pipeline/run	POST	同步执行完整 Pipeline，课堂快速验证场景直接返回 PipelineResult。
/api/v1/pipeline/submit	POST	异步提交 Pipeline 任务，需要 Celery 和 Redis。
/api/v1/pipeline/status/{job_id}	GET	查询异步 Pipeline 任务状态。
/api/v1/pipeline/recompute-corrected	POST	应用交互端人工修正后重跑 S3/S4/S5。
/api/v1/pipeline/analyze	POST	返回元件引脚定位、网表和拓扑图。
/api/v1/pipeline/compare-netlist	POST	调试接口，直接比较 reference 与 current_netlist_v2。
/api/v1/pipeline/visualize/ports	POST	返回端口映射列表，用于交互端可视化。
/api/v1/angnt/ask	POST	提交 Agent 问答任务。
/api/v1/angnt/status/{job_id}	GET	轮询 Agent 结果。
/ws/telemetry/system	WebSocket	实时推送硬件遥测帧。

6.2 PipelineRequest 与 PipelineResult

PipelineRequest 的核心字段包括 station_id、images_b64、conf、iou、imgsz、reference_id、reference_circuit、rail_assignments、port_annotations、net_role_assignments、net_alias_assignments、net_merge_assignments、pin_polarity_assignments 和 ic_annotations。交互端当前将图片转为不含 data:image 前缀的纯 base64 字符串，便于服务端统一解码。rail_assignments 用于指定电源轨角色；port_annotations 只要求用户标注输入/输出端口；net_role_assignments、alias 和 merge 主要用于高级调试或人工修正。

PipelineResult 是交互端、课堂状态和 Agent 共用的结果模型。它承载 job_id、station_id、stages、component_count、net_count、progress、similarity、diagnostics、risk_level、risk_reasons、comparison_report、runtime_metadata 等字段。stages 中的每个阶段都包含 stage 名称、数据和耗时；runtime_metadata 则记录模型路径、推理参数、参考电路来源、人工标注和网络归一化信息。

6.3 netlist_v2

netlist_v2 是当前主链路最重要的数据结构。它不是简单的文字网表，而是一个可被交互端、validator、Agent 和后续实验统计共同消费的结构化对象。其核心内容包括 board_schema_id、components、nets、node_index 和 board_topology。components 中每个元件包含 component_id、component_type、package_type、part_subtype、polarity、orientation、symmetry_group、pins、confidence 和 metadata；pins 中保留 pin_id、pin_name、hole_id、electrical_node_id、electrical_net_id、observations、confidence、is_ambiguous 和 metadata。

nets 表示由 Union-Find 合并后的电气网络，每个 ElectricalNet 包含 electrical_net_id、member_node_ids、member_hole_ids 和 power_role。node_index 保存 electrical_node_id 到 hole_id 的索引，board_topology 保存完整的 node_to_holes 和 rail_assignments。这样，交互端不仅能知道某个引脚属于哪个 net，还能在用户点击某个网络时高亮全部相关孔位。

6.4 validator_report_v2 与高亮协议

validator_report_v2 是诊断结果的核心报告格式。summary 中包含 total_item_count、logic_correct、similarity、comparison_mode、match_type、ignore_component_id、ignore_hole_id、ignore_passive_pin_order、allow_extra_wires、strict_functional_pin_roles 和 report_layers 等信息。items 则是具体诊断项，每项携带 error_code、category、severity、message、suggested_action 和 evidence_refs。

高亮协议 labguardian_highlight_v1 把 evidence_refs 转换为交互端可直接绘制的 targets。target 可以是 component_bbox_ref，渲染为元件框；可以是 pin_keypoint_ref，渲染为引脚点；也可以是 hole_candidate_ref，渲染为当前孔位、目标孔位和候选孔位。AgentService 也可以把同一份高亮协议作为 evidence 输出，使自然语言解释与可视化定位保持一致。

6.5 RuntimeEvidence 与 ContextPack

RuntimeEvidence 是 Agent 从 ClassroomState 中抽取的运行证据，包含 station_id、risk_level、error_codes、error_tags、findings、netlist_v2、circuit_snapshot、reference_circuit、current_scene_id、history_facts 和 history_summary 等。它是 Agent 的事实输入，而不是自然语言长上下文。ContextPack 则是 PCM 编译后的每轮上下文，包含 pushed_facts、allowed_tools、prompt_rules、citation_requirements 和 evidence_refs，并附带 ContextPackMetrics 记录推送事实数、工具数、证据数、历史长度和估算 token。

这样的结构使 Agent 的行为可控：它只能看到被推送的事实和被允许的工具，不会越过 validator 重新猜测孔位或网络。对于边缘端部署，这也有助于减少上下文长度，降低小模型或规则模板的负担。

第七章 关键技术与创新点

7.1 结构化事实链

项目最大的工程创新在于把“看图诊断电路”拆成多个可解释的事实转换步骤，而不是用单个端到端大模型直接回答。S1 负责元件实例，S1.5 负责引脚候选，S2 负责孔位吸附，S3 负责导通网络，S4 负责逻辑比较，Agent 负责解释。这种结构化事实链让每一步都能被单独测试、单独可视化、单独修正，也使错误定位更具体。

7.2 多视角与抗遮挡策略

针对真实实验中常见的遮挡问题，系统预留多角度拍摄与多视图融合能力。S1 支持 side recall candidates, S2 的协议包含多视图 observation、evidence_source、decisive_view_id、fusion_confidence、fusion_margin 和 cross_view_agreement。虽然当前流程以单图输入为主，但服务端数据结构和 S2 融合逻辑已经为多视角扩展预留空间。遮挡感知权重规则会在 top 不可见或置信度低时提高 side 视图权重，避免单一俯视图失效。

7.3 BoardSchema 与 Snap-to-Grid

BoardSchema 把物理孔位规范化为稳定 electrical_node_id，是连接视觉与电气逻辑的桥梁。默认比赛板 schema 覆盖 A1-E63、F1-J63 和 LP/LN/RP/RN 两段电源轨，并兼容历史 rail_top+、rail_top-、rail_bot+、rail_bot- 命名。Snap-to-Grid 不仅选取最近孔位，还结合 snap_distance、pitch、候选列表、元件几何约束和多视图投票形成可信度评估。

7.4 图论拓扑与逻辑比较

CircuitAnalyzer 使用 NetworkX 和 Union-Find 生成电气拓扑，compare_logical_graphs 则把参考电路和当前网表都转换为 component-net 二分图，再进行拓扑感知比较。相比逐孔位对比，图比较可以容忍无极性器件的引脚互换、内部信号名称差异和合理的额外导线；相比纯文本规则，它又能严格处理电源、地、功能引脚和极性器件。

7.5 PCM Agent 与可验证回答

PCM Agent 的创新在于“先压缩事实，再选择工具，最后生成回答”。它根据错误族和意图动态选择工具白名单，避免把所有技能、知识和历史都塞进上下文。Verifier 则保证输出必须引用错误码或证据，并在危险风险时输出安全提醒。该机制特别适合实验教学：学生得到的是可追溯的解释，教师也可以根据 evidence_refs 检查系统是否真的基于当前电路作答。

7.6 人工修正闭环

许多 AI 辅助系统只关注“模型一次性给出答案”，但真实教学现场需要可修正。LabGuardian 的 recompute-corrected 接口和交互端 NetlistView 让用户把 AI 识别结果修成可信事实，再让服务端重新执行拓扑和验证。这种闭环把 AI 作为助教而非裁判，既增强可用性，也符合工程教育中“学生理解并修正错误”的目标。

7.7 边缘可观测与可复现实验

项目不仅关注功能，也关注运行可观测性。`runtime_metadata` 记录模型路径、版本、参数、设备和板型；`telemetry_frame_v1` 记录 CPU、内存、iGPU、NPU 和 `pipeline_stage`。通过统一的运行元数据与遥测帧，系统可以支撑 p50/p90 延迟、峰值内存、模型版本、上下文长度、工具调用数和验证结果等实验指标，为后续复现实验和性能评估提供依据。

第八章 测试验证与质量保障

8.1 现有测试线索

项目已经建立基础回归样例与冒烟测试，覆盖板型映射、逻辑图比较、参考电路加载、Pipeline 阶段契约和 Agent 诊断流程等关键环节。现有测试能够验证主链路输入输出是否稳定，也能在功能迭代时发现数据结构漂移、错误码变化和降级逻辑失效等问题。后续测试重点应继续扩展多视图融合、孔位吸附质量、Agent ReAct 循环、硬件遥测和边缘端推理一致性。

8.2 建议的测试体系

测试层级	测试对象	重点指标
单元测试	BoardSchema、 Union-Find、 net normalization、 error code、 ContextPack	映射正确性、边界输入、空值降级、错误码稳定。
阶段测试	S1、 S1.5、 S2、 S3、 S4、 S5	阶段输入输出 schema、 duration_ms、 fallback 标记、不确定性字段。
回归测试	典型参考电路和错接 fixture	逻辑正确率、错误码一致性、 suggested_action 稳定性。
交互端测试	上传、结果展示、人工修正、Agent 轮询	状态流转、错误提示、按钮禁用、高亮目标一致性。
集成测试	交互端→服务端→Agent 端到端	完整链路可运行、 PipelineResult 和 AgentResult 可解析。
边缘测试	DK-2500 运行、模型路径、遥测、延迟	p50/p90、峰值 RSS、CPU/iGPU/ NPU 占用、降级行为。

8.3 数据集与评测建议

视觉模型评测应将元件检测、引脚检测和孔位映射分开统计。元件检测可统计 mAP、漏检率和误检率；引脚检测可统计关键点误差、可见性分类和 ordered pin 顺序正确率；孔位映射应统计 Top-1 hole 准确率、Top-k 候选覆盖率、snap_confidence 分布和低置信场景召回率。对于多视图，建议比较 top-only 与 multi-view 的 A/B 结果，重点关注遮挡、反光、强弱光、倾斜拍摄和复杂布线样本。

拓扑和诊断评测应以电路级任务为单位。可选择一阶 RC、反相运放、同相运放、NE555 定时器、LED 限流、三极管开关等教学模板，构造正确、缺线、错孔、短路、极性反接、缺保护电阻、端口标注错误等场景。指标包括逻辑正确判定准确率、错误码命中率、错误定位准确率、建议动作可执行性、误报率和漏报率。

8.4 Agent 质量保障

Agent 测试不应只看自然语言是否“像人话”，还要检查是否遵守证据约束。建议为每个 error family 构造 golden case，验证 ContextPack.allowed_tools 是否符合预期，工具调用顺序是否在白名单

内，回答是否包含至少一个当前错误码或 `evidence_ref`，`danger` 场景是否优先提醒断电与短路复查。对于概念问答、实验指导和混合意图，应检查检索源是否符合 `retrieval contract`，`wrong-scene rate` 和 `legacy fallback rate` 必须为零。

第九章 部署运行与边缘化方案

9.1 本地运行部署

服务端本地运行流程为安装运行依赖、启动 Redis、启动 Celery Worker、启动 FastAPI。课堂快速验证阶段可以直接使用同步接口 `POST /api/v1/pipeline/run`，无需异步任务；如果要测试 `/submit` 和 `/status/{job_id}`，则需要 Redis 和 Celery Worker。交互端运行流程为安装 npm 依赖后启动页面服务；若服务端不在 127.0.0.1:8000，则通过 `VITE_API_BASE_URL` 指定服务端地址。

9.2 Docker 与模型目录

边缘部署建议采用统一模型根目录，并把模型资源以只读方式挂载到容器。组件检测模型和引脚检测模型分别通过 `YOLO_MODEL_PATH` 和 `PIN_MODEL_PATH` 指定；当显式路径不可用时，系统应按预设候选路径查找，并在运行元数据中记录实际加载结果。为了保证复现实验，模型版本、推理尺寸、置信度阈值和板型 schema 应写入 `runtime_metadata`。

9.3 DK-2500 部署设想

DK-2500 搭载 Intel Core Ultra 5 225U，具备展示 CPU、iGPU、NPU 异构协同的硬件条件。当前系统已具备设备配置、`runtime_metadata`、`telemetry` 和 OpenVINO/GenAI 接入预留等基础能力；S1/S1.5 的 ONNX 导出、OpenVINO GPU/NPU 插件适配、INT8 量化、PT 与 ONNX 一致性测试、真机 p50/p90 延迟和峰值内存统计等仍属于后续边缘评测工作。

9.4 运行安全与降级策略

实验教学系统必须有降级策略。视觉模型缺失时，S1.5 可以输出 `unavailable` 外壳；校准失败时，S2 使用 `synthetic_fallback` 并标记低可信；拓扑分类器失败时，`scene_id` 为空并跳过 `fault_case` 检索；硬件遥测字段不可用时返回 `null`；Agent 没有真实 LLM 时仍使用确定性模板 `provider`。这样的 `fail-open` 或 `fail-closed` 策略能避免系统在现场因一个模块不可用而整体崩溃，同时也防止错误默认值污染诊断。

第十章 项目完成情况与边界说明

10.1 当前已形成的工程成果

- 服务端服务骨架已建立，包括 pipeline_service、guidance_service、version_service、rag_service、agent_service 和 classroom_state。
- 新网表模型 netlist_v2 已成为主输出结构，S3/S4/validator 和交互端均围绕该结构消费事实。
- 视觉主链已固定为 YOLO-Detect 组件检测和 full-image YOLO-Pose 引脚检测，OBB 和 ROI fallback 保留兼容但不作为默认主路。
- S2 已输出 components[.pins]、候选孔位、候选节点、多视图证据、不确定性和吸附质量字段。
- BoardSchema 默认电路板已支持 63 行主区和分段电源轨，用户只需标注电源轨和输入/输出端口。
- S4 已基于 logical_reference_v1 和 topology-aware graph compare 输出 validator_report_v2。
- PCM Agent 已具备 RuntimeEvidence、ContextPack、错误族路由、确定性工具、ReAct 循环、Verifier 和 repair 分支。
- Web 交互端已完成单工位诊断界面，支持上传、事实链展示、诊断卡片、Agent 对话和人工重算。
- 说明材料较完整，覆盖视觉契约、比较架构、板型格式、错误码、检索契约、边缘部署和遥测协议。

10.2 完成情况与边界说明

本阶段系统重点聚焦面包板电路的结构化诊断，已形成“图像输入 → 元件与引脚识别 → 孔位映射 → 网表生成 → 参考比较 → 诊断解释”的主链路。面向“面包板原型验证 → PCB 成品制造”的全周期教学闭环仍具有拓展价值，但 PCB AOI、VLM 微观诊断和 Anomalib 热力图等能力尚未纳入当前主链路。

这种边界收敛有利于保证最终报告的准确性。项目先完成最核心、最可验证、最能支撑教学闭环的“图像→孔位→网表→比较→解释”主链路，再将 VLM、PCB AOI 和更复杂的边缘异构加速作为未来扩展。报告中对未完成能力采取审慎表述，避免将规划性内容描述为已完成成果。

10.3 后续改进计划

阶段	目标	验收产物
近期	稳定 Pipeline 契约和交互端诊断流程，补齐关键错接场景。	更多 fixture、回归测试、交互端高亮和人工修正体验优化。
中期	完成 DK-2500 真机部署与 edge benchmark。	ONNX/OpenVINO 模型、INT8 量化实验、p50/p90/RSS/telemetry 报告。
中期	扩展 reference DSL 和教学知识库。	更多教学模板、datasheet_v2、fault_case 和 circuit_kb 条目。
后期	引入更强的多视角融合和低置信交互。	遮挡样本 A/B 测试、可视化证据 overlay、标定误差统计。

扩展	评估 VLM/PCB AOI 是否重新纳入独立模块。	独立 scope、独立 API、独立测试，不破坏主链路检索契约。
----	----------------------------	----------------------------------

第十一章 风险分析与改进计划

11.1 视觉识别风险

视觉模型的主要风险是遮挡、反光、拍摄角度、元件类别不平衡和引脚关键点误差。改进方案包括继续扩充真实实验样本，强化遮挡、强弱光和密集布线数据；对低 `snap_confidence` 和 `multi_view_vote_conflict` 的样本建立人工复核机制；将 `decisive_view_id`、`per_view_contribution` 和 `snap_distance` 可视化到交互端；对 IC、轴向元件和电位器等特殊元件继续完善几何先验。

11.2 拓扑与参考电路风险

拓扑比较依赖 `reference DSL` 和端口标注。如果参考电路定义不准确、端口标注缺失或电源轨设置错误，系统可能给出错误诊断。改进方案包括为 `reference DSL` 建立审查流程，为每个实验模板提供示意图和默认端口提示，在交互端运行前强制检查 `rail_assignments` 和必要端口，提供 `compare-netlist` 调试接口帮助教师验证参考电路。

11.3 Agent 与知识检索风险

Agent 的风险主要是 `wrong-scene` 检索、旧知识源污染、自由发挥和安全提示缺失。当前 `retrieval contract` 已经通过合法源清单、禁用源清单、训练部署不变量和 `fail-closed` 机制降低风险。后续应继续执行 `gold set` 评测，监控 `wrong-scene rate`、`legacy fallback rate`、`datasheet recall` 和回答证据覆盖率；新增任何检索入口都必须更新 `retrieval-contract.md` 并补测试。

11.4 边缘部署风险

边缘部署风险包括模型体积、推理延迟、共享内存压力、OpenVINO 设备兼容、NPU 驱动路径不稳定和容器权限限制。改进方案包括拆分 `server-dev`、`edge-cpu`、`edge-openvino` 镜像；导出 `S1/S1.5 ONNX` 并做 `parity test`；记录阶段 `latency`、`p50/p90`、`peak RSS`、模型版本和 `board_schema_id`；对 `telemetry sampler` 采用 `defensive` 设计，避免硬件采样异常影响主服务。

11.5 交互端体验风险

交互端需要避免把复杂 JSON 直接暴露给学生。建议将 `NetlistView` 的高级修正入口分层隐藏，普通学生优先看到“哪里错、为什么、怎么改”；教师或维护者再展开 `candidate holes`、`evidence_refs`、`raw JSON` 和 `manual net roles`。对于错误卡片，应提供一键高亮、修复步骤、参考图示和安全提醒；对于低置信结果，应明确提示“需人工确认”，避免误导。

第十二章 应用价值、推广路径与实施组织

12.1 对学生学习过程的价值

LabGuardian 对学生最大的价值是把实验反馈从“等待教师巡查”转变为“即时自助排错”。在传统课堂中，学生经常在某个接线点卡住十几分钟，最终得到的反馈可能只是教师一句“这里接错了”。这种反馈虽然能解决眼前问题，却不一定帮助学生理解面包板导通关系、元件引脚功能和电路拓扑。LabGuardian 的诊断结果包含元件、引脚、孔位、节点、网络、参考连接和建议动作，学生可以沿着证据链理解错误产生原因，从而把一次排错转化为一次概念强化。

系统还可以支持形成性评价。学生每次上传和修正都会产生结构化数据，例如常见错误码、错孔类型、短路风险、端口标注错误、低置信孔位和修复前后相似度变化。这些数据可以帮助学生回顾自己的薄弱环节，也可以作为实验报告中的过程性证据。与只看最终波形是否正确相比，过程数据更能反映学生是否真正理解了电路搭建逻辑。

12.2 对教师教学组织的价值

对教师和助教而言，LabGuardian 可以把大量重复检查工作前置给系统。教师不再需要逐个实验台检查“电阻是否跨行”“LED 是否有限流电阻”“电源轨是否短路”“运放正负电源是否接反”等基础问题，而是可以优先处理系统标记为 `danger` 或 `warning` 的工位。课堂状态中保存的 `risk_level`、`diagnostics`、`comparison_report`、`netlist_v2` 和 `topology_label` 可以进一步汇总为教师看板，显示哪些实验模板最容易出错、哪类元件最常被误接、哪些学生需要重点辅导。

这种看板并不是为了替代教师，而是帮助教师把注意力从机械巡检转向概念讲解和个性化指导。当某一错误码在多个工位频繁出现时，教师可以暂停全班操作，统一讲解对应知识点；当某个工位多次出现低 `snap_confidence` 或多视图冲突时，教师可以指导学生调整拍摄角度或复查遮挡区域。

12.3 对课程资源建设的价值

LabGuardian 的 `reference DSL`、`logical_reference_v1`、`teaching_scene`、`fault_case`、`datasheet_v2` 和 `circuit_kb` 可以逐步沉淀为课程知识资产。每个实验模板不仅包含电路原理，还包含参考拓扑、端口语义、预期测量点、常见故障、错误码解释和安全提醒。随着课程迭代，教师可以不断补充新的参考电路和故障案例，使系统从单次辅助工具发展为电子实验课程的数字化资源库。

这类资源库具有可复用性。不同学校的电子基础实验虽然教材和器件略有差异，但面包板导通规则、两脚无极性元件、LED 限流、运放供电、三极管引脚等知识具有通用性。只要 `BoardSchema`、参考 DSL 和 `datasheet` 条目支持版本化，系统就能迁移到不同课程、不同实验箱和不同板型。

12.4 项目实施组织建议

从工程实施角度，团队可按照“视觉算法、服务端架构、交互端交互、知识库与测试、边缘部署”五条线并行推进。视觉算法线负责数据采集、标注规范、YOLO-Detect/YOLO-Pose 训练、孔位吸附误差分析和多视图融合；服务端架构线负责 Pipeline 契约、`BoardSchema`、`netlist_v2`、`validator` 和服务层接口；交互端交互线负责上传、证据链展示、人工修正、Agent 对话和教师看板；知识库与测试

线负责 reference DSL、fault_case、datasheet_v2、golden tests 和检索契约；边缘部署线负责 DK-2500 环境、模型转换、OpenVINO、telemetry 和 benchmark。

后续实施中应坚持“先明确契约，再调整实现，再补充测试，再更新说明材料”的原则。例如修改 S2 输出字段时，应同步更新阶段协议与回归样例；修改错误码时，应同步更新诊断规则与测试样例；修改 Agent 检索行为时，应同步检查场景路由、知识源约束和 fallback 逻辑。这样可以降低多人协作时的数据结构漂移风险。

12.5 推广路径与落地成本

推广路径可以分为三个阶段。第一阶段是单工位验证，目标是跑通一套典型实验，如 RC 电路、LED 限流或运放反相放大器，展示上传图片、生成网表、发现错误和 Agent 解释。第二阶段是课程实验室试点，目标是在若干实验台部署边缘服务器或共享服务器，采集真实错接样本，优化视觉模型和交互端交互。第三阶段是课程资源平台化，目标是形成参考电路库、故障案例库、教学概念库和教师看板，把系统嵌入实验教学流程。

落地成本主要包括边缘计算设备、摄像设备或学生手机、模型训练数据、教师维护参考电路的时间，以及系统部署维护成本。当前设计不依赖固定工业相机，优先支持学生手机上传，这有利于降低硬件门槛。服务器端若能在 DK-2500 本地运行，则可以减少云资源费用和网络依赖。长期看，最大的成本不是硬件，而是高质量数据和课程知识资产建设，因此项目应重视数据标注规范、知识条目版本化和教师可编辑工具。

12.6 阶段性交付清单

交付类别	交付内容	验收标准
功能交付	图像上传、Pipeline 运行、网表展示、诊断卡片、Agent 解释、人工重算。	至少一套典型实验可稳定端到端运行。
数据交付	参考电路、错接样本、datasheet JSON、fault_case、测试 fixture。	每个模板覆盖正确与典型错误场景。
算法交付	组件检测、引脚检测、孔位映射、拓扑比较、风险分类。	输出字段符合契约，错误码和证据可追溯。
部署交付	服务端启动脚本、交互端启动脚本、模型目录、Docker/Redis/Celery 配置。	新环境能依据说明材料复现系统运行。
材料交付	项目报告、接口说明、测试说明、部署说明、验收材料。	说明材料与实际系统能力一致，不夸大未纳入主链路的功能。

12.7 典型课堂使用流程

典型课堂使用流程可设计为“正常电路—错误电路—人工修正—Agent 解释—边缘遥测”五步。第一步展示一个正确搭建的参考电路，上传图片后让系统输出较高相似度和无严重错误报告，说明系统能理解正确拓扑。第二步将某个电阻或导线移动到错误孔位，重新上传后展示 HOLE_MISMATCH、NODE_MISMATCH 或 COMPONENT_SHORTED_SAME_NET 等错误码，并点击诊断卡片让交互端高亮具体引脚和孔位。第三步在 NetlistView 中人工修正低置

信孔位或补充端口标注，调用 `recompute-corrected`，说明系统支持人机协同而不是一次性黑盒判断。第四步打开 `AgentChat`，让系统用自然语言解释错误原因、证据和修复步骤，强调回答引用的是 `validator_report_v2` 与 `netlist_v2` 事实。第五步展示 `runtime_metadata` 或 `telemetry`，说明系统为 DK-2500 边缘部署和异构计算展示预留了可观测数据。

项目验收展示应避免将所有技术点一次性堆叠，而应围绕“问题为何困难、系统如何拆解、当前完成了什么、下一步如何验证”展开。可以先用一张面包板实物图解释遮挡和孔位问题，再用一张数据流图解释 S1 到 S4 事实链，然后展示交互界面中的 `evidence chain`，最后说明 `Agent` 为什么不会凭空猜测。对于 PCB AOI 和 VLM 微观诊断，可以说明其属于后续扩展方向；当前主线聚焦面包板拓扑诊断，工程边界更清晰，验收目标也更可控。

在课堂试点中，使用流程可转换为学生操作流程：学生先按实验指导书搭建电路，使用手机拍摄上传；系统返回风险等级和错误位置；学生依据建议修改电路；修改后再次上传验证；最终将诊断前后对比、修复说明和测量结果写入实验报告。教师则可以在后台查看班级错误统计，选择共性问题进行集中讲解。这种流程把 `LabGuardian` 嵌入已有教学环节，而不是要求课程完全重构。

第十三章 结论

LabGuardian 选题具有明确的教学价值和工程挑战。它面向高校电子实验中真实存在的痛点：教师巡检压力大、学生排错慢、面包板遮挡严重、二维原理图到三维实物映射困难、诊断结果缺少证据链。项目并没有停留在“用 AI 看一张图片”的层面，而是把视觉识别、孔位映射、板型知识、图论拓扑、参考电路比较、诊断报告、交互端高亮和 Agent 解释串联成一条完整事实链。

从当前完成情况看，项目最扎实的部分是结构化 Pipeline 和数据契约。S1/S1.5/S2 把图像转为 pin/hole 事实，S3 通过 BoardSchema 和 Union-Find 生成 netlist_v2，S4 通过 topology-aware graph compare 生成 validator_report_v2，Agent 通过 RuntimeEvidence 和 ContextPack 在证据约束下回答问题。交互端则提供了较完整的诊断闭环，支持上传、展示、诊断、对话和人工修正。

后续工作的重点应是以真实样本和真机数据验证系统可靠性，而不是继续扩大概念范围。建议优先完成多视角样本评测、错接 fixture 扩充、交互高亮完善、DK-2500 边缘评测和知识库 gold set 评估。在此基础上，再评估是否将 VLM 微观诊断和 PCB AOI 作为独立扩展模块纳入。总体而言，LabGuardian 已具备从实验系统走向教学工具原型的基础，若能持续加强数据、测试和边缘部署验证，将有潜力成为高校电子实验智能化教学的有价值样板。

参考资料

1. 《LabGuardian：基于边缘 AI 的智能实验助教系统》项目设计方案书。
2. 电子类基础实验课程教学大纲与实验指导书。
3. 面包板电路搭建、元器件引脚识别与电路拓扑分析相关资料。
4. 边缘人工智能、目标检测、关键点检测、知识增强检索与智能体编排相关技术资料。
5. 项目实验记录、系统测试记录与阶段性验收材料。

附录：术语与模块说明

术语	含义
hole_id	面包板物理孔位编号，如 A12、LP31。
electrical_node_id	面包板静态导通节点编号，由 BoardSchema 根据 hole_id 解析。
electrical_net_id	动态电气网络编号，由元件和导线连接合并后生成。
netlist_v2	当前主链路的结构化网表格式，包含元件、引脚、网络和板型拓扑。
logical_reference_v1	教师或参考模板提供的逻辑参考电路格式。
validator_report_v2	S4 输出的结构化诊断报告，包含错误码、证据和建议。
RuntimeEvidence	Agent 从课堂状态抽取的当前运行证据。
ContextPack	PCM 编译后的最小上下文包，包含事实、工具、规则和证据引用。
BoardSchema	描述面包板孔位、导通节点、电源轨和别名的板型规则。
Snap-to-Grid	将视觉关键点吸附到最近面包板孔位并计算置信度的过程。

模块	说明
视觉感知模块	完成元件检测、引脚关键点识别和多视图候选证据生成。
孔位映射模块	将图像关键点吸附到面包板孔位，并映射为电气导通节点。
拓扑重构模块	基于板型规则和导线连接关系生成 netlist_v2 与拓扑图。
参考比较模块	将当前电路与逻辑参考电路进行图结构比较，输出诊断报告。
Agent 诊断模块	基于运行证据、知识库和确定性工具生成可解释反馈。
交互展示模块	提供上传、结果展示、错误定位、人工修正和对话交互能力。
边缘遥测模块	采集 CPU、内存、iGPU、NPU 和 Pipeline 阶段状态，用于性能观程。